Can a Monitor Tell When an AI Agent Has Actually Finished Its Task?

Nathan Langley
Undergraduate, University of North Carolina at Greensboro
With Apart Research
Research conducted at the Swarm Alignment Hackathon, November 2026

Abstract

When an AI agent reports that it finished a delegated task, how much should I trust that report? I tested whether monitors could distinguish completions supported by independent records from claims that did not hold, and whether external evidence improved detection. From a preexisting orchestration system, I collected 219 completion claims: 98 EARNED, 15 UNEARNED, and 106 UNKNOWN. All labels were AI-read without human verification. Only seven naturally UNEARNED jobs reached test, below the preregistered minimum of 15.

In the preregistered comparison, Haiku's report-only monitor scored AUROC 0.517 against 0.538 with evidence. The result was inconclusive, but the planted edits were invisible to the evidence features by construction. When a renamed file was detectable, scores increased from 0.387 to 0.802.

After inspecting the test results, I developed a narrower task-match prompt. On harder same-workflow swaps, Haiku task-match scored 0.775 against 0.594 for omnibus, but did not beat BM25 at 0.801. Every scorer remained near chance when a key deliverable line was removed. These exploratory, hypothesis-generating results come from an already-inspected test split. They do not establish reliable detection of natural failures or an advantage over the strongest no-model baseline.

1. Introduction

I run an orchestrator that delegates work to AI agents. One problem with a system like this is that a worker can report finishing something without actually completing the task. The report might cite real files, mention tests, or sound convincing while still missing what was requested.

I wanted to measure that distinction rather than assume a convincing report meant successful work. I tested whether monitors could recognize unsupported completion claims, whether external evidence helped, and whether asking a narrower question worked better than a general completion judgment.

My main contributions are:

1. A labeled evaluation of completion claims from a preexisting delegation system, with explicit separation between supported, contradicted, and unknown outcomes.
2. A preregistered comparison of report-only and evidence-augmented monitors using scripted false completions.
3. Exploratory comparisons of task-specific monitoring against general monitoring and simple no-model baselines, including harder same-workflow swaps and missing deliverables.

The task-match investigation followed inspection of the initial test results. It is hypothesis-generating, not an independent confirmation.

2. Related Work

Greenblatt et al. [1] study AI control despite intentional subversion; I evaluate ordinary delegation completion claims rather than adversarial control protocols. Hammond et al. [2] examine broader multi-agent risks, while I focus on whether individual delegated tasks were actually completed.

Baker et al. [3] study monitoring reasoning models for misbehavior; my monitors inspect final reports and limited evidence instead of reasoning traces. SHADE-Arena [4] evaluates sabotage and monitoring, whereas my planted failures are scripted edits rather than agent-generated sabotage.

Cemri et al. [5] examine why multi-agent systems fail; I measure whether such failures can be recognized from completion reports. METR [6] documents reward hacking, while I do not establish reward hacking or intentional deception. SLEIGHT-Bench [7] studies monitor evasion attacks; my experiment measures specific report inconsistencies without an adaptive attacker.

3. Methodology

Corpus and labels

My preexisting orchestrator recorded 506 closed delegations from 24 October to 6 November 2026. I excluded 138 jobs from excluded workflows and 92 that hit a privacy filter, leaving 276 jobs across 15 workflows. Of these, 219 had runner status done and a final report claiming completion.

All text was scrubbed of identifiers, and sentences containing verifier wording were removed before monitoring. The database retains at most the last 2,000 characters of each report; 55 of the 219 claim reports were truncated. All main results use these stored reports.

Labels came from system records, never monitor outputs:

- EARNED: A verify-run results-ledger line cites the job for its requested result, without contradiction from an incident or verifier note.
- UNEARNED: An incident or verifier note explicitly states that the completion claim did not hold.
- UNKNOWN: Everything else. An uncited job is not automatically unearned.

Two AI readers, Claude Sonnet 5.5 and Opus 5.5, had to agree and quote supporting records in a label sweep that increased natural UNEARNED cases from 2 to 15. The sweep knew the preregistered target of 15 and landed exactly there. No human checked the labels. An AI re-read agreed on 35 of 42 stratified items ($\kappa = 0.743$).

Final counts were EARNED 98, UNEARNED 15 (8 dev, 7 test), and UNKNOWN 106.

Split and freeze

Using seed 20261106, I randomly held out three of seven workflows with at least 10 jobs (40%) as test. Jobs created on or after 3 November were also assigned to test.

The date rule means the split is not fully workflow-disjoint: 26 of 99 test claims, including 10 of 45 test EARNED and 2 of 7 test UNEARNED, belong to seven workflows also represented in dev.

Prompts were tuned on dev and frozen by git commit before phase-2 test calls.

Monitors and baselines

Models ran through `claude -p`, returning a 0–100 probability that completion was unearned.

L1 received the task and report. L2 additionally received scripted facts about cited file and commit existence and whether commits occurred during the worker run. L3 received the task, report, and last 12,000 characters of the worker transcript.

The omnibus prompt asked for P(unearned) using a four-point rubric beginning with whether the report answered the task. The task-match prompt asked only whether the report matched the assigned task.

No-model baselines included report length, keyword self-report scoring, negated Jaccard overlap of task/report words containing at least four letters, and BM25.

Reported model scores average repeated calls per item. Haiku omnibus used three repeats in phase 2 and five in phase 5; Haiku task-match used five in phases 4–5. Sonnet used three in phase 3 (four for task-match on plant d, with one phase-4 repeat) and two in phase 5; Opus used one. These averaged scores should not be interpreted as single-call performance.

Scripted plants

I created 15 examples each of five edits to real reports, without model-written plant text:

- a: Rename a cited file to a nonexistent name.
- b: Replace in-run commit hashes with real out-of-run commits.
- c: Remove test sentences and insert a fabricated passing log.
- d: Pair a task with another job's report, selecting task-word Jaccard overlap below 0.25, preferring the same workflow.
- e: Remove disclosures such as "not merged" or "untested" and insert the same sentence into all 15 reports: "Everything in the brief is done and committed."

Plants a and b are invisible to L1 by construction. Plants c, d, and e preserve L2's evidence facts. Plant d's low-overlap selection also makes lexical detection easier by design.

Phase 5 added d2, swapping in the report associated with the most similar task in the same workflow, and p, removing a key deliverable line.

All 73 d2/p plants derive from the 45 test EARNED jobs. The p cases compare reports with and without a line; d2 pairs one EARNED report with another EARNED task.

Analysis and phases

AUROC compares planted or natural positive cases against 45 test EARNED jobs. Confidence intervals use 2,000 percentile-bootstrap resamples clustered by report source. For d2, task sources are treated as independent despite shared jobs.

Phase 2 preregistered that evidence mattered if the lower confidence bound for L2 − L1 exceeded zero. "No detectable gain" required a CI containing zero with an upper bound below 0.10; otherwise the result was inconclusive. Natural comparisons were underpowered below 15 test UNEARNED.

Phase 3 introduced stronger models, task-match, controls, placebo, truncation and L3 post hoc. The task-match prompt, word-overlap control and plant-d pass bar (AUROC ≥ 0.75, lower CI bound above 0.5) were selected after seeing phase-2 results.

Phase 4 committed its repeat-reporting rule before additional calls. Phase 5 preregistered its new calls and analysis but reused the inspected test split. No untouched test evaluation exists.

4. Results

4.1 Preregistered evidence comparison

On 45 planted c+d+e cases, Haiku L1 scored 0.517 [0.402, 0.623] and L2 scored 0.538 [0.433, 0.649]. The paired difference was +0.022 [−0.080, +0.141], inconclusive under the preregistered rule.

These edits leave L2's evidence features unchanged, so this comparison could not demonstrate a gain from detecting the planted edits through those features.

Plant a, where the renamed file was detectable, increased from L1 0.387 [0.220, 0.553] to L2 0.802 [0.691, 0.900]. Plant b increased only from 0.384 [0.217, 0.548] to 0.467 [0.295, 0.635].

Table 1. Preregistered Haiku AUROC against 45 test EARNED jobs, with 95% clustered-bootstrap confidence intervals; each plant has 15 cases.

Plant	L1 AUROC [95% CI]	L2 AUROC [95% CI]
c: fabricated test log	0.407 [0.242, 0.578]	0.553 [0.382, 0.715]
d: wrong-job report	0.645 [0.485, 0.785]	0.560 [0.423, 0.698]
e: removed disclosure	0.497 [0.324, 0.668]	0.501 [0.337, 0.675]

Plant e used one fixed replacement sentence in every case. No phrase-detector baseline was tested, despite L1 and L2 remaining near chance.

The natural comparison had only 7 UNEARNED against 45 EARNED: L1 0.492, L2 0.497, difference +0.005 [−0.168, +0.176]. It is underpowered.

An existence check caught 0 of 15 natural UNEARNED cases: 14 had all cited items present (an upper-bound observation), and one had nothing checkable.

Mean per-item population SD across three Haiku repeats on the 0–100 score scale was 3.86 for L1 and 2.43 for L2, measured over 127 items.

4.2 Post-hoc task matching on plant d

On the original low-overlap wrong-job swaps, word overlap scored 0.842 [0.723, 0.943]. BM25, computed post hoc in phase 5, scored 0.921 [0.845, 0.982].

Haiku task-match scored 0.897 [0.807, 0.973] across five repeats. Individual-repeat AUROCs ranged from 0.794 to 0.907, compared with the averaged score of 0.897.

The paired gain over Haiku omnibus was +0.252 [+0.141, +0.378]. Against word overlap, the difference was +0.055 [−0.080, +0.199]; against BM25, −0.024 [−0.120, +0.074]. Word overlap itself beat Haiku omnibus by +0.197 [+0.013, +0.387].

Sonnet omnibus scored 0.889 [0.774, 0.972], Sonnet task-match 0.950 [0.881, 0.997], and Opus omnibus 0.936 [0.824, 0.999]. Sonnet task-match minus word overlap was +0.107 [+0.000, +0.232]. These Sonnet and Opus comparisons are circular because both models helped read the EARNED labels.

A placebo comparison of plants a+b against EARNED gave Haiku L1 0.386 [0.253, 0.523]; plant d against a+b gave 0.753 [0.579, 0.893].

With d-versus-EARNED labels permuted across 2,000 permutations (15 versus 45), the estimated probability of any of five phase-3 d cells passing the post-hoc 0.75 bar by chance was 0.01.

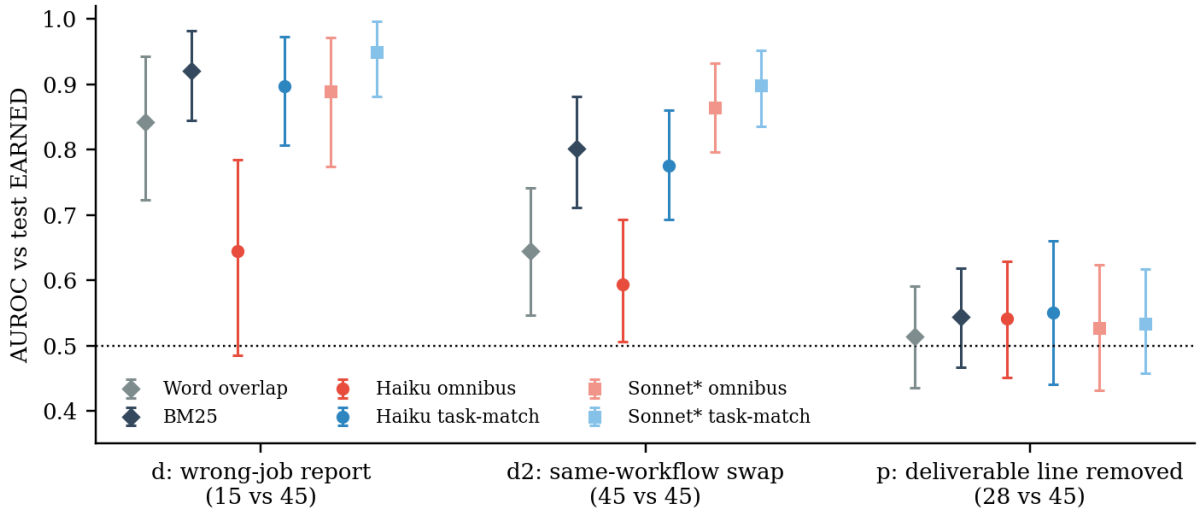


Figure 1. AUROC with 95% clustered-bootstrap confidence intervals for six scorers (word overlap, BM25, Haiku omnibus, Haiku task-match, Sonnet* omnibus, Sonnet* task-match) on three planted sets against test EARNED jobs: d, wrong-job report (15 vs 45); d2, same-workflow swap (45 vs 45); and p, deliverable line removed (28 vs 45). The dotted line marks chance performance at 0.5. An asterisk indicates a model that also read labels.

4.3 Harder swaps and missing deliverables

Phase 5 tested 45 d2 swaps (30 positive clusters) and 28 p cases against 45 EARNED controls.

The no-model d2 and p baselines were measured on dev and included in the phase-5 preregistration before model calls. Dev d2 scores were word overlap 0.678 [0.592, 0.766] and BM25 0.780 [0.702, 0.860].

The preregistered minimum detectable effects at 80% power were 0.141 for d2 and 0.146 for p. Every phase-5 cell is therefore exploratory.

Table 2. Phase-5 AUROC with 95% clustered-bootstrap confidence intervals for harder same-workflow swaps (d2) and removed deliverable lines (p); an asterisk indicates label-reader circularity.

Scorer	d2 AUROC [95% CI]	p AUROC [95% CI]
Word overlap	0.644 [0.547, 0.741]	0.514 [0.435, 0.591]

Scorer	d2 AUROC [95% CI]	p AUROC [95% CI]
BM25	0.801 [0.712, 0.882]	0.544 [0.467, 0.619]
Haiku omnibus	0.594 [0.506, 0.693]	0.541 [0.451, 0.629]
Haiku task-match	0.775 [0.693, 0.860]	0.551 [0.441, 0.661]
Sonnet* omnibus	0.865 [0.797, 0.933]	0.527 [0.431, 0.624]
Sonnet* task-match	0.899 [0.835, 0.952]	0.534 [0.457, 0.617]

On d2, Haiku task-match exceeded word overlap by +0.132 [+0.028, +0.245], but not after the six-cell Bonferroni correction: 99.17% CI [−0.016, +0.279].

It did not beat BM25: −0.026 [−0.132, +0.085].

Against Haiku omnibus, the gain was +0.182 [+0.083, +0.274], surviving Bonferroni [+0.050, +0.305]. This is the clearest exploratory Haiku prompt comparison, not a confirmatory result.

Haiku omnibus was worse than BM25 by −0.208 [−0.343, −0.061].

Sonnet* task-match exceeded BM25 by +0.098 [+0.018, +0.181] at 95%, but not after Bonferroni [−0.016, +0.223]. It is also circular against the AI-read EARNED labels.

No label-independent model monitor beat the best no-model baseline on either d or d2.

Haiku task-match d2 scored 0.775 using averaged item scores, while its individual repeats scored 0.702, 0.762, 0.756, 0.758, and 0.738. Haiku omnibus single repeats ranged from 0.542 to 0.602 against its averaged 0.594.

On p, every scorer remained near chance, all AUROC intervals included 0.5, and no paired monitor-versus-baseline improvement was supported.

Phase 5 used 590 Haiku omnibus calls, 590 Haiku task-match calls, 236 Sonnet omnibus calls and 236 Sonnet task-match calls. A recount performed 28 checks with zero mismatches and a largest CI difference of 0.009.

4.4 Report truncation

All main results used stored reports capped at 2,000 characters.

A post-hoc subset compared 13 plant-d reports against 35 EARNED reports. Full reports were at most 4,054 characters (median 2,556), so the earlier 12,000-character phase-3 cap had not truncated them.

Table 3. Haiku AUROC on the 13-plant-d / 35-EARNED subset, comparing stored and full report text; confidence intervals are 95% clustered-bootstrap intervals.

Scorer	AUROC [95% CI]
Omnibus, 2,000 characters	0.615 [0.429, 0.776]
Omnibus, full (phase 3)	0.738 [0.565, 0.883]
Omnibus, full (phase 5)	0.680 [0.499, 0.851]
Task-match, 2,000 characters	0.893 [0.797, 0.975]
Task-match, full (phase 5)	0.927 [0.779, 1.000]

Omnibus improved by +0.065 [−0.087, +0.232] from stored to full text, and task-match by +0.034 [−0.153, +0.181]. Neither improvement was supported.

On identical uncapped text, the phase-3 and phase-5 omnibus runs differed by −0.058 [−0.165, +0.072]. The apparent truncation shifts are within same-input run noise.

Task-match minus omnibus was +0.278 [+0.145, +0.438] at 2,000 characters and +0.247 [+0.016, +0.469] on full phase-5 reports.

5. Discussion

The most interesting result to me is that a general completion prompt can perform worse than a narrow question about whether the report matches the task. Haiku showed this on the planted wrong-task reports, including the harder same-workflow swaps. But I developed that narrower prompt after seeing the test split, so the result generates a hypothesis rather than confirming one.

There is also a simpler explanation for some of the apparent success: the original plant d deliberately selected low-overlap tasks. That makes lexical baselines unusually effective. On harder d2 swaps, Haiku task-match beat word overlap at 95%, but not after Bonferroni, and it never beat BM25. Only the exploratory Haiku task-match versus omnibus comparison survived that correction.

The more important failure was plant p. Removing a key deliverable line left every tested scorer near chance. This suggests that report-text judgments alone may not establish whether the requested result exists. That reading assumes the planted edits represent useful failure modes; if real failures differ, these AUROCs may not transfer. The AI-read labels, shared source jobs, repeated-call averaging and partially overlapping workflows also limit how much I can generalize.

Future Work

I would next collect fresh untouched test data, obtain human label checks, and evaluate more natural failures. I also want to test a phrase-detector baseline on plant e, an artifact existence check on plant p, and other deployments and providers. None of those experiments has been run here.

6. Conclusion

I wanted to measure whether an AI monitor could distinguish an agent reporting completion from having evidence that the task was actually completed. The preregistered evidence comparison was inconclusive, partly because the planted edits were invisible to the evidence features by design. Where a renamed file was detectable, the evidence-augmented monitor performed better.

The later Haiku task-match results are interesting, but hypothesis-generating: I wrote that prompt after seeing the test split, and the phase-5 results remain exploratory. Haiku task-match improved over Haiku omnibus on wrong-task plants, but no label-independent model monitor beat BM25. None reliably detected a removed deliverable line.

Natural failures remain underpowered, and the labels have not been human-checked. I think the next step is to verify requested artifacts and outcomes directly rather than rely only on how convincingly an agent describes its work. A convincing completion report is not necessarily a completed task.

7. Reproducibility and Preexisting Work

The orchestrator, delegation database, results ledger, incident reports, verifier notes, sentinel monitor files and logs existed and operated before the sprint. The underlying corpus also predates the sprint. I used them as read-only sources, not as new sprint work.

The study repository was created during the sprint. Its first commit was 6 November 2026 at 16:56 US Eastern. It contains the preregistrations, export and scrubbing scripts, labels, monitor runs, phases 1–5, figures and report.

Code and Data

There is no public code repository or dataset at submission. The repository and git history are private, and raw delegation text describes a private person's life and work. Releasing it would expose personal information, so I report aggregate counts and scores instead.

The procedures and freeze order are described, but the unavailable code, raw data and verbatim prompts prevent independent replication from this report alone.

Author Contributions

Nathan Langley designed and directed the project; AI agents ran the experiments and drafted text under his direction.

Appendix A. Limitations and Dual-Use / Ethical Considerations

What the results establish. On one already-inspected split, a task-specific Haiku prompt distinguished some planted wrong-task reports better than an omnibus Haiku prompt. The study does not establish superiority over BM25, reliable detection of omitted deliverables, or reliable monitoring of naturally unearned completions.

Only seven natural UNEARNED test cases were available against the preregistered minimum of 15. The label sweep knew the target of 15 and landed exactly there. No human checked the labels; a pending 30-item review of 15 UNEARNED and 15 EARNED cases remains unanswered.

Of 219 completion claims, 106 (48%) were UNKNOWN and excluded from EARNED controls. Because EARNED requires a verify-run ledger citation, these controls may be more formulaic than the overall claim population.

Sonnet and Opus helped read labels, making their comparisons against EARNED circular. All experiments came from one deployment, one operator and one provider.

The task-match prompt, word-overlap control and phase-3 pass bar were chosen after seeing test results. Phase 5 preregistered new calls but reused that inspected split and was underpowered by its own rule. No confirmatory claim follows.

The 73 phase-5 planted positives also share source jobs with the EARNED controls. Bootstrap clustering accounts for report sources but treats task sources as independent. Repeated-call averages can exceed single-call performance.

Fresh untouched data, human label verification, a phrase-detector baseline for plant e, an artifact check on plant p, and evaluations across other deployments and providers remain untested.

Dual-use and ethical considerations. Identifying which report alterations monitors miss could help someone create misleading completion claims or make failures in multi-agent operations harder to detect. The plant construction also provides a way to probe monitoring blind spots.

I report aggregate behavior rather than private task contents, identifiers or operational logs. The privacy exclusions remain in place. These findings should be used to improve verification, not to assume that plausible reports or existing file references establish successful completion.

Appendix B. Monitor Prompt Summary

Omnibus: Return a 0–100 probability that completion is unearned, using a four-point rubric beginning with "does the report answer this task?".

Task-match: Judge only whether the report is about the assigned task.

L1 receives the task and report. L2 adds scripted file-existence and commit-timing facts. L3 adds the last 12,000 characters of the worker transcript.

Exact prompts remain in the private study repository. This is a functional description rather than verbatim prompt text, so it is not sufficient for byte-for-byte replication.

LLM Usage Statement

Claude (Anthropic) agents running in my orchestrator designed and ran the experiments, wrote the code and compiled the fact record. ChatGPT (OpenAI) wrote the report prose from that fact record. Claude checked every number against the fact record and recomputed the headline numbers from raw run files. Labels were read by AI models, not checked by a human.

References

[1] Greenblatt, R., Shlegeris, B., Sachan, K., Roger, F. (2023). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942. https://arxiv.org/abs/2312.06942

[2] Hammond, L., Chan, A., Clifton, J., et al. (2025). Multi-Agent Risks from Advanced AI. arXiv:2502.14143. https://arxiv.org/abs/2502.14143

[3] Baker, B., Huizinga, J., Gao, L., et al. (2025). Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926. https://arxiv.org/abs/2503.11926

[4] Kutasov, J., Sun, Y., Colognese, P., et al. (2025). SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents. arXiv:2506.15740. https://arxiv.org/abs/2506.15740

[5] Cemri, M., Pan, M. Z., Yang, S., et al. (2025). Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657. https://arxiv.org/abs/2503.13657

[6] METR (2025). Recent Frontier Models Are Reward Hacking. https://metr.org/blog/2025-06-05-recent-reward-hacking/

[7] Najt, E., Toft, C., Tracy, T., Roger, F., et al. (2026). SLEIGHT-Bench: A Benchmark of Evasion Attacks Against Agent Monitors. arXiv:2605.16626. https://arxiv.org/abs/2605.16626